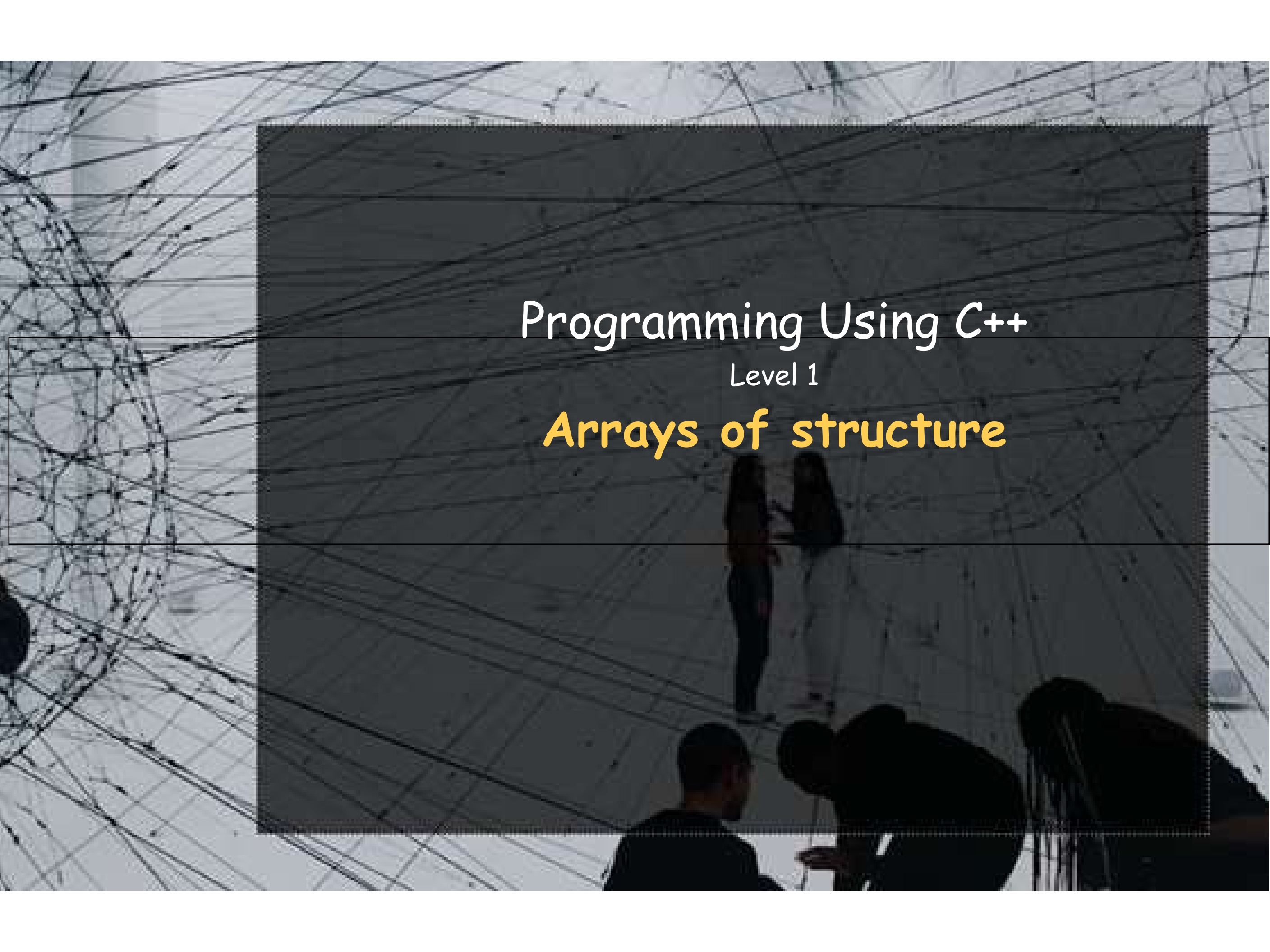


Programming Using C++ Level 1

Arrays of structure



Programming Using C++

Level 1

Arrays of structure

Definition

- ◆ An array of structures is an array where each element is a structure object.
- ◆ It allows us to store multiple records that have the same structure.
- ◆ Each element contains all the members defined inside the structure.

Explanation

- ◆ A structure defines the type of information we want to store.
- ◆ An array of that structure allows us to create multiple objects with the same set of members.
- ◆ For example, if `strInfo` contains `Name`, `Age`, and `City`, an array of `strInfo` can store information for several people.

Steps in main()

1 Define the structure

- ◆ Create a structure containing the required members.

2 Declare an array of the structure

- ◆ Create an array where each element is one structure object.

3 Access an element

- ◆ Use an index to select a specific structure from the array.

4 Access a member

- ◆ Use the dot operator . to access a member of that structure.

5 Store or modify information

- ◆ Assign values to the members of each structure element.

💡 Example

◆ Structure:

```
struct strInfo
{
    string Name;
    int Age;
    string City;
};
```

◆ Array of structures:

```
strInfo People[3];
```

◆ Access the first person's name:

```
People[0].Name
```

◆ Access the second person's age:

```
People[1].Age
```

◆ Access the third person's city:

```
People[2].City
```

➡ Result

◆ The array can store information for 3 people, where each person has the same members:

```
People[0] → Name, Age, City
```

```
People[1] → Name, Age, City
```

```
People[2] → Name, Age, City
```

Note

- ◆ The array index identifies which structure you are accessing.
- ◆ The dot operator identifies which member of that structure you want to access.
- ◆ `People[0].Name` means the Name member of the first structure in the array.

⊘ Warning

- ◆ Do not confuse the array index with the structure member.
- ◆ `People[1]` selects the second structure, while `People[1].Age` accesses its *Age* member.



Comparison

Array of Structures

- ✓ Stores multiple objects with the same structure.
- ✓ Each element can contain multiple related values.
- ✓ Useful for storing records such as people, employees, or students.

Normal Array

- ✓ Stores multiple values of one data type.
- ✓ Each element normally represents one value.
- ✓ Useful for collections such as grades, numbers, or names.

✦✦ Summary

- ✓ An array of structures stores multiple objects of the same structure type.
- ✓ Each array element can contain several related members.
- ✓ The array index and dot operator are used together to access specific data.

Thank you 😊